

STQM 270 Final Project

Brianna Sharp, Daniyal Anwar, Gage Powell, Erik Sorensen

Ferris State University

STQM 270: Introduction to Data Mining

Dr. Elies Kouider

April 29, 2026

that produced only two clusters, such as the TwoStep model, tended to group Versicolor and Virginica together, which may be useful only in scenarios where a binary distinction is sufficient.

It is also recommended that data standardization continue to be used in future analyses. The MinMax normalization applied in this project ensured that all variables were on the same scale, which is critical for distance-based clustering methods like K-Means and Kohonen. Without this step, variables with larger numerical ranges could disproportionately influence cluster formation.

Although clustering provided useful insights into natural groupings within the data, there were limitations observed in separating Versicolor and Virginica. Future work should consider combining clustering with supervised learning techniques to improve classification accuracy. For example, after identifying clusters, a classification model could be trained to better distinguish between overlapping groups.

Finally, visual analysis using scatterplots proved highly effective in interpreting clustering results. It is recommended that visualization continue to be used alongside statistical measures such as cluster quality, as it provides a clearer understanding of how well the models are performing and where overlap occurs.

Supervised Learning

Objectives

Lastly, there is the Supervised Learning model. In this model, we used the BostonHousing.xlsx dataset. We used this dataset to configure this model in the IBM SPSS

Modeler. Supervised Learning uses inputs to find predictive models, to then find the corresponding outputs. The outcome for this stream is to predict the median value of the owner-occupied home using the CAT.MEDV as the target variable.

Description

For this BostonHousing dataset, there are multiple variables that are shown in the models that should be understood before starting. As mentioned before, we now know the target variable and the purpose of this stream. Next, we need to know what the variables in the dataset mean.

“CRIM” means the crime rate per capita by town. “ZN” is the proportion of residential land zoned for lots over 25,000 square feet. “INDUS” means the proportion of non-residential business acres per town. “CHAS” means the Charles River dummy variable. “NOX” is the nitric oxide concentration. “RM” is the average number of rooms per home. “AGE” means the age of the home built prior to 1940. “DIS” means the weighted distances to five of the main Boston employment centers. “RAD” is the index of accessibility to highways. “TAX” is the full-value property-tax rate per \$10,000. “PTRATIO” is the teacher-student ratio per town.

“B-100(BK-.63)², where BK is the proportion of African Americans per town. “LSTAT” is the percentage of the lower status (lower income) of the population. Lastly, “MEDV” is the median value of the owner-occupied homes in \$10,000’s, which is the target. For the stream to find this target, we use six different trees and another model. We used Neural Networks, Quest, CHAID, Tree-AS, Random Trees, and CART. In the stream, we used the dataset and partitions before we added these models, so they can read the data the right way.

Data Mining Session

Preview from Project_BostonHousing.xlsx Node (15 fields, 10 records)

File Edit Generate

Table Annotations

	CRIM	ZN	IND...	CH...	NOX	RM	AGE	DIS	R...	TAX	PTRATIO	B	LSTAT	ME...	CAT. MEDV
1	0.027	0...	7.070	0...	0.469	7.185	61...	4.967	...	24...	17.800	392...	4.030	3...	1.000
2	0.088	1...	7.870	0...	0.524	6.012	66...	5.561	...	31...	15.200	395...	12...	2...	0.000
3	0.145	1...	7.870	0...	0.524	6.172	96...	5.950	...	31...	15.200	396...	19...	2...	0.000
4	0.170	1...	7.870	0...	0.524	6.004	85...	6.592	...	31...	15.200	386...	17...	1...	0.000
5	0.638	0...	8.140	0...	0.538	6.096	84...	4.462	...	30...	21.000	380...	10...	1...	0.000
6	0.784	0...	8.140	0...	0.538	5.990	81...	4.258	...	30...	21.000	386...	14...	1...	0.000
7	0.726	0...	8.140	0...	0.538	5.727	69...	3.796	...	30...	21.000	390...	11...	1...	0.000
8	1.232	0...	8.140	0...	0.538	6.142	91...	3.977	...	30...	21.000	396...	18...	1...	0.000
9	0.988	0...	8.140	0...	0.538	5.813	10...	4.095	...	30...	21.000	394...	19...	1...	0.000
10	0.750	0...	8.140	0...	0.538	5.924	94...	4.400	...	30...	21.000	394...	16...	1...	0.000

The above shows that the BostonHousing dataset is being read and previewed.

Project_BostonHousing.xlsx

Preview Refresh

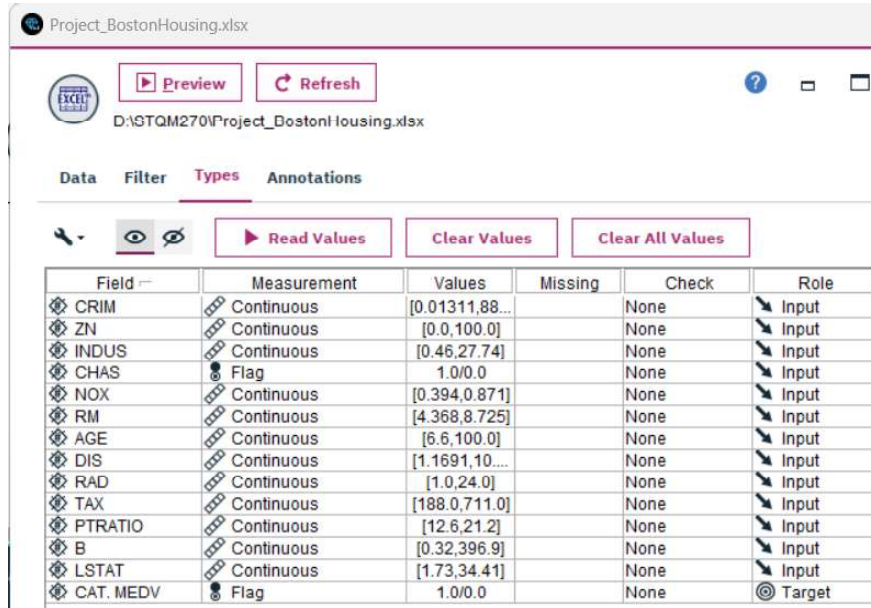
D:\STQM270\Project_BostonHousing.xlsx

Data Filter Types Annotations

Fields: 15 in, 1 filtered, 0 renamed, 14 out

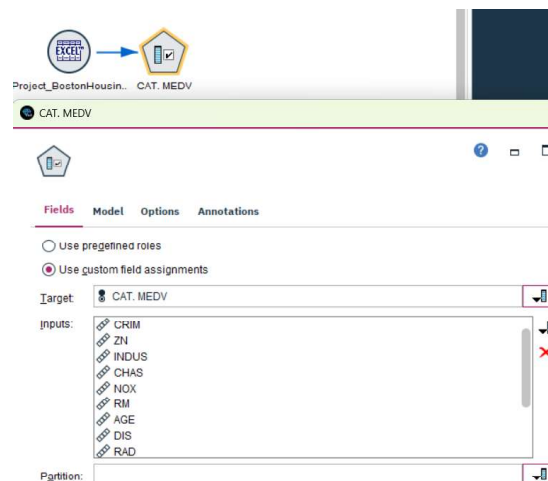
Field	Filter	Field
CRIM	→	CRIM
ZN	→	ZN
INDUS	→	INDUS
CHAS	→	CHAS
NOX	→	NOX
RM	→	RM
AGE	→	AGE
DIS	→	DIS
RAD	→	RAD
TAX	→	TAX
PTRATIO	→	PTRATIO
B	→	B
LSTAT	→	LSTAT
MEDV	✗	MEDV
CAT. MEDV	→	CAT. MEDV

The headers are then filtered. The MEDV is not counted.



Field	Measurement	Values	Missing	Check	Role
CRIM	Continuous	[0.01311,88...		None	Input
ZN	Continuous	[0.0,100.0]		None	Input
INDUS	Continuous	[0.46,27.74]		None	Input
CHAS	Flag	1.0/0.0		None	Input
NOX	Continuous	[0.394,0.871]		None	Input
RM	Continuous	[4.368,8.725]		None	Input
AGE	Continuous	[6.6,100.0]		None	Input
DIS	Continuous	[1.1691,10....		None	Input
RAD	Continuous	[1.0,24.0]		None	Input
TAX	Continuous	[188.0,711.0]		None	Input
PTRATIO	Continuous	[12.6,21.2]		None	Input
B	Continuous	[0.32,396.9]		None	Input
LSTAT	Continuous	[1.73,34.41]		None	Input
CAT. MEDV	Flag	1.0/0.0		None	Target

Next, the data types were read. The CHAS needs to be read as a Flag and the CAT.MEDV needed to be changed to a flag, then the target role in order to be properly read.



Next is creating the Feature Selection. The CAT.MEDV is renamed to “Feature Selection.” Then in the Fields tab, there, selected is the “Use Custom Field Assignments” is selected, and the Target is CAT.MEDV. In the Inputs, all measures are selected, except for CAT.MEDV. Next, we move onto the Model tab, which is not pictured. The “Maximum

Percentage of Missing Values” is set at 30.0. The rest in this tab is kept at its default. The Options tab does not change. Then we run it to get the Feature Selection node pictured below.



The above is what the model looks like so far. Next, we will run the model.

Feature Selection

File Generate Preview

Model Summary Annotations

☒ ☐ ☒ Rank

Rank	Field	Measurement	Importance	Value
1	RM	Continuous	Important	1.0
2	LSTAT	Continuous	Important	1.0
3	PTRATIO	Continuous	Important	1.0
4	ZN	Continuous	Important	1.0
5	INDUS	Continuous	Important	1.0
6	TAX	Continuous	Important	1.0
7	NOX	Continuous	Important	1.0
8	RAD	Continuous	Important	0.995
9	AGE	Continuous	Important	0.985
10	B	Continuous	Important	0.969
11	DIS	Continuous	Important	0.968
12	CRIM	Continuous	Marginal	0.942

Selected fields: 12 Total fields available: 13

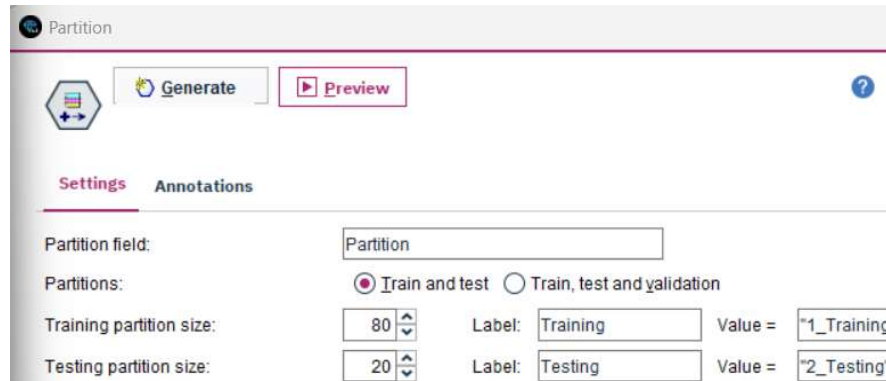
☒ > 0.95 ☒ <= 0.95 ☒ < 0.9

1 Screened Fields

Field	Measurement	Reason
CHAS	Flag	Single category too large

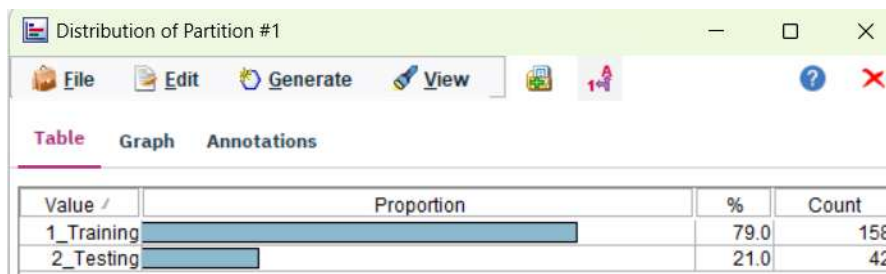
Once the model is run, the values will look like this (pictured above). The “CRIM” box will be unchecked, but we will check the box. The “CHAS” is screened.

On the model, we will attach a Partition node. We will change the “Training Partition Size” to 80, the “Testing Partition Size” to 20, and the rest will stay at its default, as shown in the screenshot below.



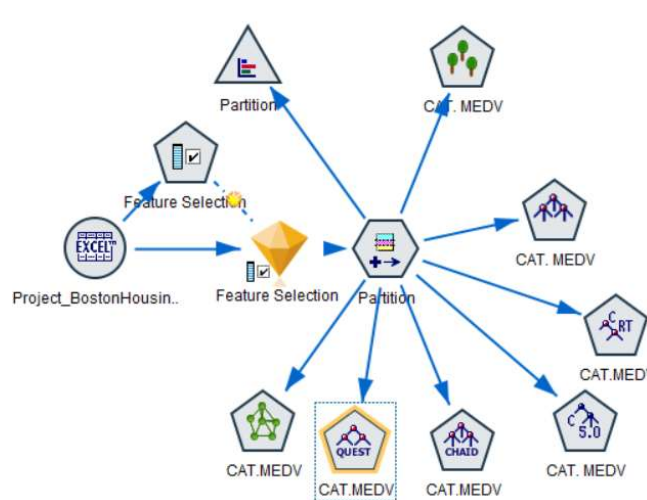
The screenshot shows the 'Partition' tool interface. It has a 'Settings' tab selected. The 'Partition field' is set to 'Partition'. The 'Partitions' section has two radio buttons: 'Train and test' (selected) and 'Train, test and validation'. The 'Training partition size' is set to 80, with a label 'Training' and a value of '1_Training'. The 'Testing partition size' is set to 20, with a label 'Testing' and a value of '2_Testing'.

Once we get the above, we will connect it to a Distribution Graph where the Field in the Plot tab is a Partition. After this, we increased the Partition Training size to 85 and the Testing size to 15 to get closer to the results we were looking for.

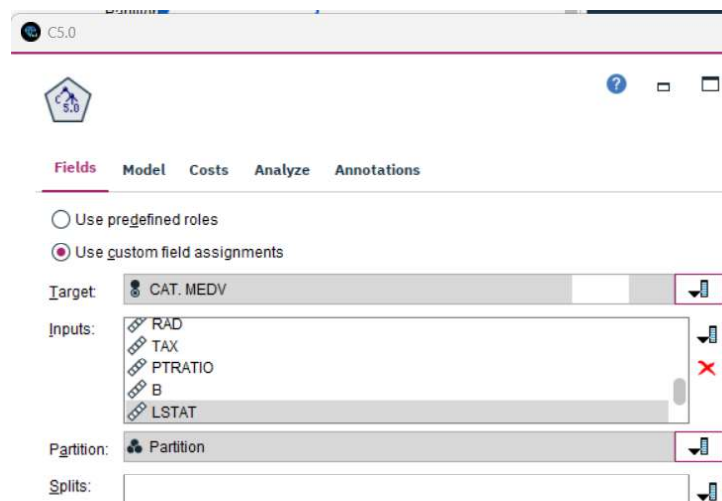


After changing the Partition Training and Testing sizes, we received the results that we were happy with.

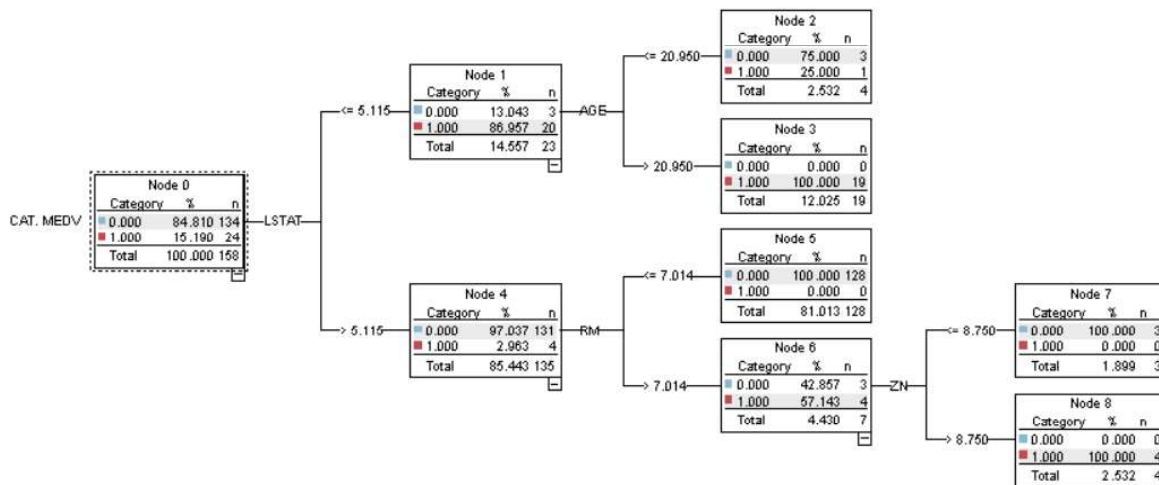
Next, we added six new decision trees: Quest, CHAID, C5.0, C&R Tree, Tree-AS, and Random Trees. In addition to that, we also included a Neural Network. Pictured below is what the steam looks like so far.



After we add our models, we need to configure them. We first rename the models by what name they are. Then we start with the C5.0 Model.



For the Fields tab in the C5.0 Model, we make the CAT.MEDV as the Target and all inputs but the target and the Partition. The Partition is the partition. So far, we leave the Model, Costs, and Analyze tabs as their defaults and run it.



Once you run the model and go to the Viewer, you will get this tree. This gives us information, but it does not give us the accuracy that we need to collect. In order to collect them, we need to attach an Analysis Node to the Model. Then you need to edit the node to have checked “Coincidence Matrices (for symbolic targets) in order to run it.

Analysis of [CAT.MEDV]

File Edit

Analysis Annotations

[-] Collapse All [+ Expand All

Results for output field CAT.MEDV

Comparing SC-CAT.MEDV with CAT.MEDV

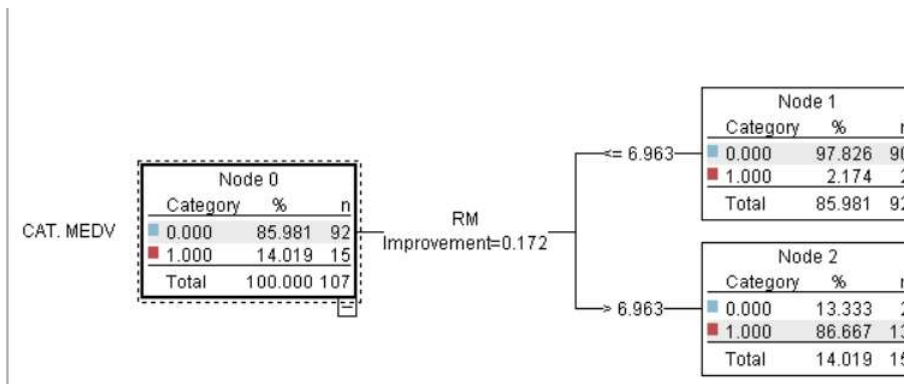
Partition	1_Training	2_Testing
Correct	157 99.37%	38 90.48%
Wrong	1 0.63%	4 9.52%
Total	158	42

Coincidence Matrix for SC-CAT.MEDV (rows show actuals)

Partition	0.000000	1.000000
0.000000	134	0
1.000000	1	23
Partition = 2_Testing	0.000000	1.000000
0.000000	35	0
1.000000	4	3

When you run it, you then get the accuracy. As you can see, the Training Accuracy is 99.37%, and the Testing Accuracy is 90.48%. The overfit of this equals 8.89.

The next model to create is CART. We leave everything as its default and run it to get its model node.



Once you run it and go to the Viewer tab, you get the tree as shown above. Then we attached the Analysis node to the CART Model node to find the accuracy.

Analysis of [CAT. MEDV] #3

File Edit

Analysis Annotations

[-] Collapse All [+ Expand All

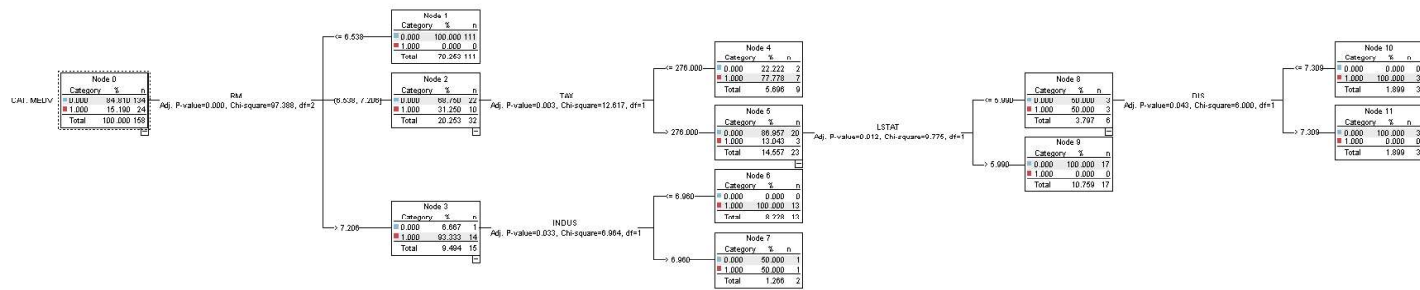
Results for output field CAT. MEDV

Comparing \$R-CAT. MEDV with CAT. MEDV

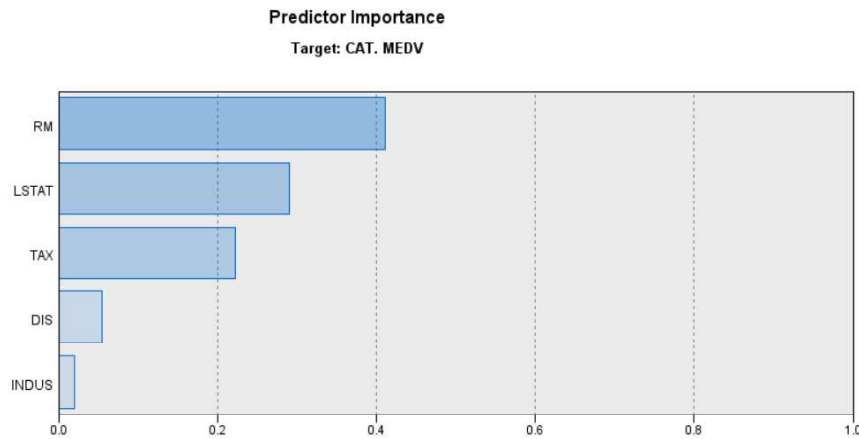
'Partition'	1_Training		2_Testing	
Correct	149	94.3%	38	90.48%
Wrong	9	5.7%	4	9.52%
Total	158		42	

Once you run the Analysis, you get the above results. These show that the Training accuracy is 94.3% and the Testing accuracy is 90.48%. You then can put these in the Excel chart, recording the accuracies plus the overfit. Using basic math, you can find that the overfit for the CART model is 3.82. While the testing accuracy of C5.0 and CART is the same, CART is a better model than the C5.0 model because of the overfitting.

The next model that we will look at is CHAID. In the Editing part of the CHAID node, we leave everything as default and run it to get its model node. Running this model node will get you a bigger tree than the last two models, as shown below.



In addition to this tree, a graph shows what is the most important for the target, as shown below.



This shows that the measure RM is more important than LSTAT, and LSTAT is more important than TAX. After we found this, we attached the Analysis node to the CHAID Model node and ran it to get the accuracies for this model, as shown below.

Results for output field CAT. MEDV

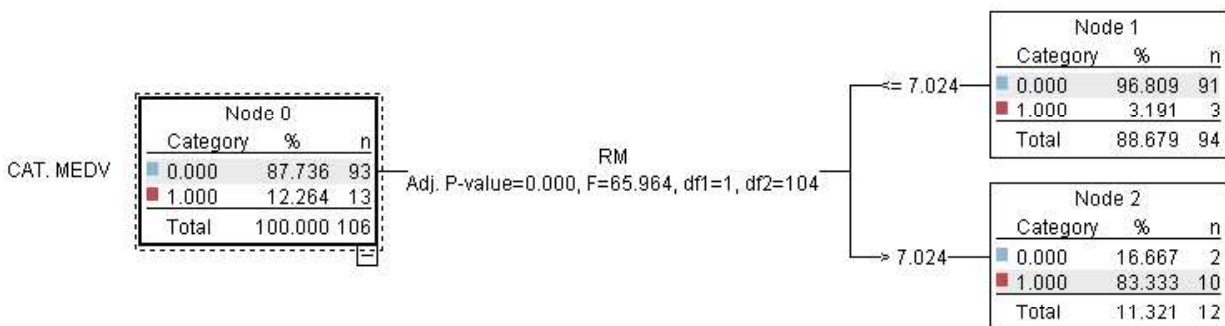
Comparing \$R-CAT. MEDV with CAT. MEDV

'Partition'	1_Training		2_Testing	
Correct	155	98.1%	36	85.71%
Wrong	3	1.9%	6	14.29%
Total	158		42	

This shows that the Training Accuracy for the CHAID model is 98.1% and its Testing Accuracy is 85.71%. Calculating this into our Excel sheet of model accuracies, we are able to find the overfit is 12.39. This shows that this model is not as good as our last two models. You

can edit the model node to include the likelihood to get slightly better data; however, it is still not the greatest model.

The fourth model we will work on is Quest. In this Edit portion of this node, we will keep all of its defaults and run it as-is. Once we run it, we get the Quest model node, and we run that to get its results, as shown below.



As you can see, this tree is much smaller than our previous model. After we find these results, we are going to attach the Analysis node to the Model node to get the accuracies, as shown below.

Results for output field CAT. MEDV				
Comparing \$R-CAT. MEDV with CAT. MEDV				
'Partition'	1_Training		2_Testing	
Correct	148	93.67%	38	90.48%
Wrong	10	6.33%	4	9.52%
Total	158		42	

This shows that the model's Training accuracy is 93.67% and its Testing accuracy is 90.48%. Filling these accuracies into the Excel sheet, you get an overfit of 3.19. Because this model has the smallest overfit, it is the best model so far. However, to get it even more accurate,

we are going to edit Quest to have “Enhance model stability (bagging).” This gives us our new results, shown below.

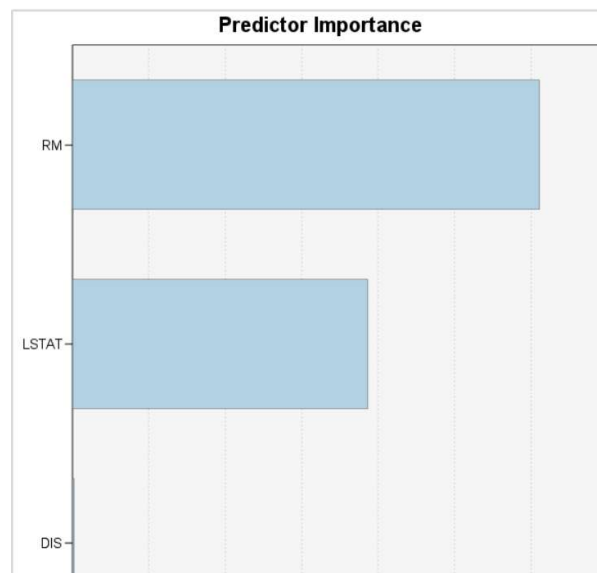
Results for output field CAT. MEDV

Comparing \$R-CAT. MEDV with CAT. MEDV

'Partition'	1_Training		2_Testing	
Correct	155	98.1%	40	95.24%
Wrong	3	1.9%	2	4.76%
Total	158		42	

Calculating the 98.1% Training and 95.24% Testing accuracies in Excel, we get 2.86 for the Overfit. This is the best model in this stream so far.

The next model we use is Tree-AS. So far, we will run the node as it is with its defaults. This node takes a little bit longer to execute compared to the past model nodes. Running the model node shows shocking results in the Predictor Importance, as shown below.



While the above data is hard to fit because of its size, it shows the dramatic results that RM and LSTAT are much more important than any other measure. After finding these results, we attached the Analysis node to find the accuracies, shown below.

Results for output field CAT. MEDV

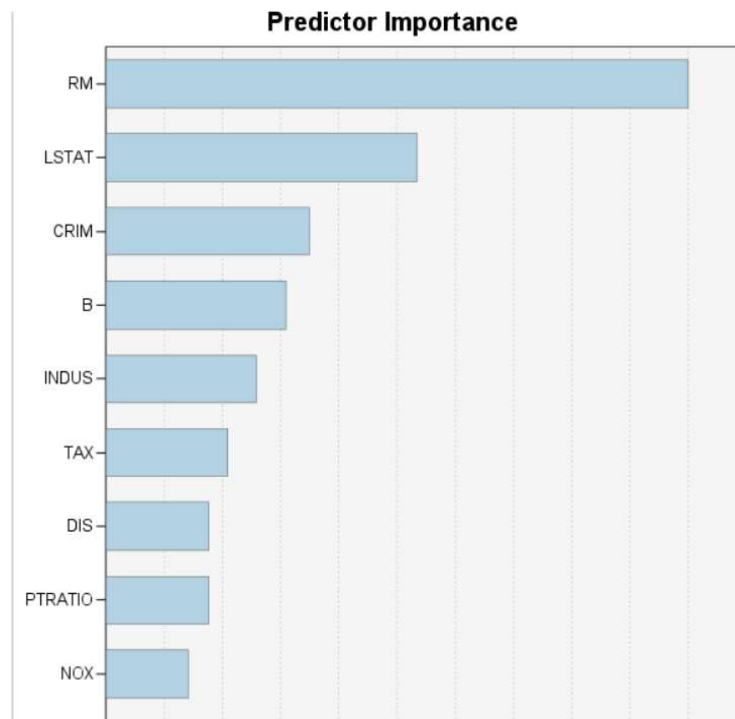
Comparing \$R-CAT. MEDV with CAT. MEDV

'Partition'	1_Training		2_Testing	
Correct	153	96.84%	38	90.48%
Wrong	5	3.16%	4	9.52%
Total	158		42	

As you can see, the Training accuracy is 96.84%, and the Testing accuracy is 90.48%.

Filling these in the Excel sheet, you get an Overfit of 6.36. This is the third highest of the Overfits so far, so this is not the best model so far.

The next model we look at is Random Trees. In the Build Options tab of the editing portion of this node, we change the “Number of Models to Build” to 20. We leave the rest as its default and run it to get the model node and its results, shown below.



For these node results, the Predictor Importance variables increased compared to the last model. After we find this, we are then going to attach the Analysis node to it to get the accuracies as shown below.

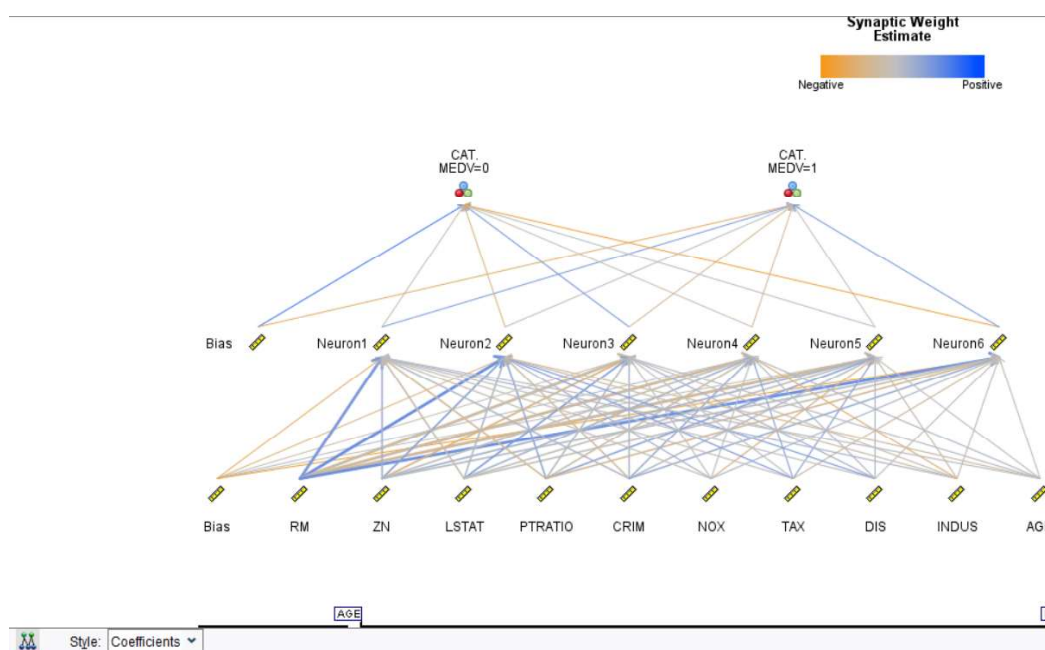
Results for output field CAT. MEDV

Comparing \$R-CAT. MEDV with CAT. MEDV

'Partition'	1_Training		2_Testing	
Correct	156	98.73%	40	95.24%
Wrong	2	1.27%	2	4.76%
Total	158		42	

This shows that for the Random Trees model, the Training accuracy is at 98.73% and the Testing accuracy is at 95.24%. Calculating these totals in our Excel sheet, we get a 3.48 Overfit. This shows that the model is the second-best one so far.

The next model we look at is the Neural Network. So far, we are going to run it with its defaults. Below shows what the model running looks like.



The above shows one of the graphics that the model nodes run. This shows the Neural Network using Coefficients. It also shows RM is most important and Age is least important. Another graphic this model shows is the accuracy. After we look at this node, we are going to attach the Analysis node to this model node and run it to find the accuracy.

■ Results for output field CAT. MEDV

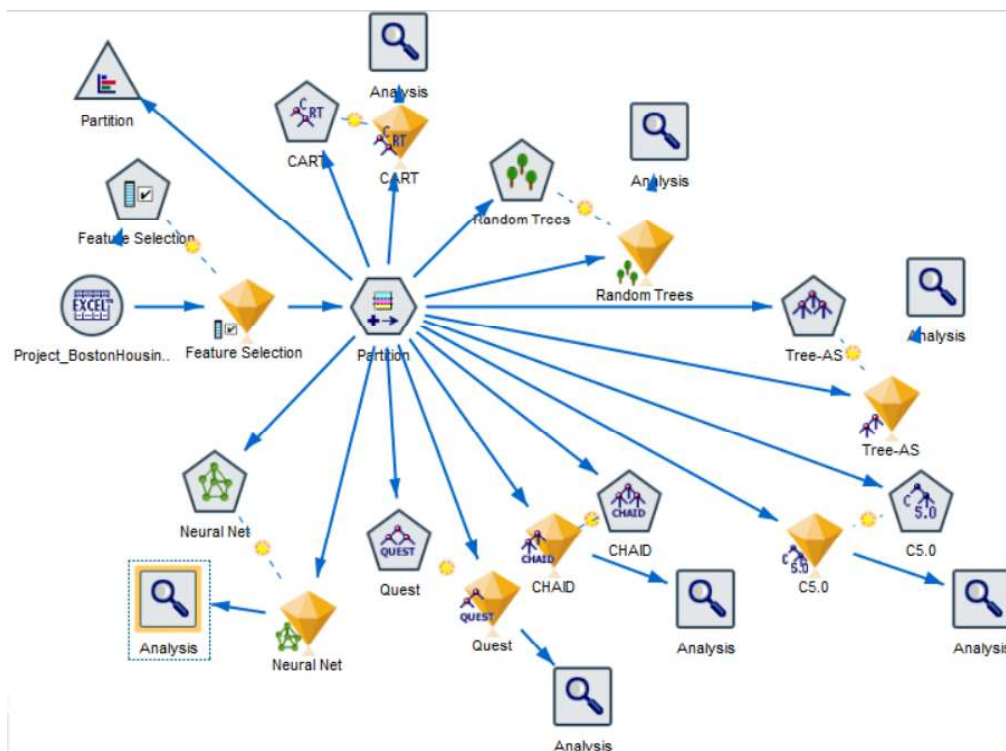
■ Comparing \$N-CAT. MEDV with CAT. MEDV

'Partition'	1_Training		2_Testing	
Correct	151	95.57%	39	92.86%
Wrong	7	4.43%	3	7.14%
Total	158		42	

The above shows the Neural Network accuracies. It shows that the Training accuracy is 95.57% and the Testing accuracy is 92.86%. When you fill these in in the Excel sheet, you get an Overfit of 2.71. The conclusion is that the Neural Network model is the best model for the stream. The second-best model is the Quest model.

Results

Below shows what the final outcome of the stream is.



This shows how the dataset is connected to the models and how they could be potentially formatted. This gives us crucial information on how we get our results from the data generally.

The outcomes of the above stream show different Testing and Training accuracies, along with overfitting. To find Overfit, you need to find the difference between Training and Testing Accuracy. The overfit will give you the answer to which model is the best to show what affects the median price of a Boston home. This sheet, which reflects this, is shown below.

Model	Training Accuracy	Testing Accuracy	Overfit
C5.0	99.37	90.48	8.89
Quest	98.1	95.24	2.86
CHAID	98.1	85.71	12.39
Random Trees	98.73	95.25	3.48
CART	94.3	90.48	3.82
Tree-AS	96.84	90.48	6.36
Neural Net	95.57	92.86	2.71

The above shows the results of the stream. This shows that the best model to show our target was the Neural Network, then Quest, then Random Trees, then CART, Tree-AS, C5.0, and lastly CHAID. CHAID is not a good example for this stream. In most of the models, the variables that showed up the most were “RM,” “LSTAT,” “CRIM,” “DIS,” and “TAX.” This means that the number of rooms a house has, the amount of lower-income people in the area, the crime rate, the distance between employment, and the property tax value are the highest things to look for when looking at the median price of a home in Boston.

Recommendations

If a client is looking to find out which home has the highest median value, then they should look for a home with the most rooms in an area where people have a high income, a low crime rate, is close to employment, and where the property taxes are higher. If the client was looking for a lower median price of a home, then they should look for the opposite of that.